

MI MIPITX API

Version 2.01

© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.00	Initial release	12/16/2019
2.01	Added new API: MI_ MipiTx_InitDev and MI_ MipiTx_DeInitDev	05/08/2020

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. 概述 1

- 1.1. 模块说明 1
- 1.2. 流程框图 1
- 1.3. 关键字说明 1

2. API 参考 2

- 2.1. MI_MipiTx_CreateChannel 3
- 2.2. MI_MipiTx_DestroyChannel 4
- 2.3. MI_MipiTx_GetChannelAttr 5
- 2.4. MI_MipiTx_StartChannel 5
- 2.5. MI_MipiTx_StopChannel 6
- 2.6. MI_MipiTx_SetTimingConfig 7
- 2.7. MI_MipiTx_GetTimingConfig 8
- 2.8. MI_MipiTx_InitDev 8
- 2.9. MI_MipiTx_DeInitDev 9

3. Mipitx 数据类型 10

- 3.1. MI_MipiTx_TimingConfig_t 11
- 3.2. MI_MipiTx_LaneNum_e 12
- 3.3. MI_MipiTx_ChannelSwapType_e 13
- 3.4. MI_MipiTx_ChannelAttr_t 13
- 3.5. MI_MipiTx_InitParam_t 14

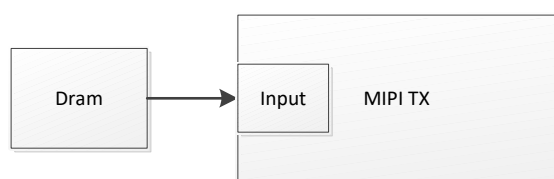
4. 错误码 16

1. 概述

1.1. 模块说明

将 **Dram** 数据通过 **MIPI** 接口协议发送出去。

1.2. 流程框图



1.3. 关键字说明

无。

2. API 参考

该功能模块提供以下 API:

API 名	功能
MI_MipiTx_CreateChannel	创建一个 MipiTx channel
MI_MipiTx_DestroyChannel	销毁一个 MipiTx channel
MI_MipiTx_GetChannelAttr	获取 MipiTx channel 属性
MI_MipiTx_StartChannel	启用 MipiTx channel
MI_MipiTx_StopChannel	禁用 MipiTx channel
MI_MipiTx_SetTimingConfig	设置 MipiTx 通道时序参数
MI_MipiTx_GetTimingConfig	获取 MipiTx 通道时序参数
MI_MipiTx_InitDev	初始化 MipiTx 设备
MI_MipiTx_DeInitDev	反初始化 MipiTx 设备

2.1. MI_MipiTx_CreateChannel

➤ 描述

创建一个 MipiTx channel。

➤ 语法

```
MI_S32 MI_MipiTx_CreateChannel(MI_U32 u32ChannelId, MI_MipiTx_ChannelAttr_t
*pstMipiTxChAttr);
```

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入
pstMipiTxChAttr	MipiTx channel 属性指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

- 不支持重复创建

➤ 举例

mipi tx 功能设置如下

```
MI_U32 u32ChnId = 0;
MI_MipiTx_ChannelAttr_t stMipiChnAttr;
memset(&stMipiChnAttr, 0x0, sizeof(MI_MipiTx_ChannelAttr_t));

stMipiChnAttr.eLaneNum = E_MI_MIPITX_LANE_NUM_4;
stMipiChnAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
stMipiChnAttr.u32Dclk = 445500;
stMipiChnAttr.u8DCLKDelay = 0;
stMipiChnAttr.u32Width = 1920;
stMipiChnAttr.u32Height = 1080;
stMipiChnAttr.peChSwapType = NULL;

MI_MipiTx_CreateChannel(u32ChnId, &stMipiChnAttr);

MI_MipiTx_TimingConfig_t stTimingCfg;
memset(&stTimingCfg, 0x0, sizeof(MI_MipiTx_TimingConfig_t));

stTimingCfg.u8Lpx = 0x07;
```

```
stTimingCfg.u8ClkHsPrpr = 0x08;
stTimingCfg.u8ClkZero = 0x20;
stTimingCfg.u8ClkHsPre = 0x01;
stTimingCfg.u8ClkHsPost = 0x10;
stTimingCfg.u8ClkTrail = 0x0a;
stTimingCfg.u8HsPrpr = 0x01;
stTimingCfg.u8HsZero = 0x0F;
stTimingCfg.u8HsTrail = 0x0A;

MI_MipiTx_SetTimingConfig(u32ChnId, &stTimingCfg);

MI_MipiTx_StartChannel(u32ChnId);

/****exit****/
MI_MipiTx_StopChannel(u32ChnId);
MI_MipiTx_DestroyChannel(u32ChnId);
```

注：Dclk 参数设置参考 [MI_MipiTx_ChannelAttr_t](#) 中的注意；
Timing 参数设置参考 [MI_MipiTx_TimingConfig_t](#) 中的注意。

➤ 相关主题

[MI_MipiTx_DestroyChannel](#)

2.2. MI_MipiTx_DestroyChannel

➤ 描述

销毁一个 MipiTx channel。

➤ 语法

```
MI_S32 MI_MipiTx_DestroyChannel(MI_U32 u32ChannelId);
```

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \text{ 成功。} \\ \text{非 } 0 \text{ 失败，参照} \text{ [错误码](#)。} \end{array} \right.$

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)

➤ 相关主题

[MI_MipiTx_CreateChannel](#)

2.3. MI_MipiTx_GetChannelAttr

➤ 描述

获取一个 MipiTx channel 属性。

➤ 语法

```
MI_S32 MI_MipiTx_GetChannelAttr(MI_U32 u32ChannelId,  
MI\_MipiTx\_ChannelAttr\_t *pstMipiTxChAttr);
```

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入
pstMipiTxChAttr	MipiTx channel 属性指针。	输出

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)

➤ 相关主题

[MI_MipiTx_CreateChannel](#)

2.4. MI_MipiTx_StartChannel

➤ 描述

启用 MipiTx channel.

➤ 语法

MI_S32 MI_MipiTx_StartChannel(MI_U32 u32ChannelId);

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)。

➤ 相关主题

[MI_MipiTx_StopChannel](#)

2.5. MI_MipiTx_StopChannel

➤ 描述

禁用 MipiTx channel。

➤ 语法

MI_S32 MI_MipiTx_StopChannel(MI_U32 u32ChannelId);

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	u32ChannelId 号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)。

➤ 相关主题

[MI_MipiTx_StartChannel](#)

2.6. MI_MipiTx_SetTimingConfig

➤ 描述

设置 MipiTx 通道时序参数。

➤ 语法

```
MI_S32 MI_MipiTx_SetTimingConfig(MI_U32 u32ChannelId, MI\_MipiTx\_TimingConfig\_t
*pstMipiTimingCfg);
```

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入
pstMipiTimingCfg	MipiTx 时序参数。	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)。

➤ 相关主题

[MI_MipiTx_GetTimingConfig](#)

2.7. MI_MipiTx_GetTimingConfig

➤ 描述

获取 MipiTx 通道时序参数。

➤ 语法

```
MI_S32 MI_MipiTx_GetTimingConfig(MI_U32 u32ChannelId,  
MI\_MipiTx\_TimingConfig\_t *pstMipiTimingCfg);
```

➤ 参数

参数名称	描述	输入/输出
u32ChannelId	MipiTx channel 号。	输入
pstMipiTimingCfg	MipiTx 时序参数	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

无

➤ 举例

请参见 [MI_MipiTx_CreateChannel](#)。

➤ 相关主题

[MI_MipiTx_SetTimingConfig](#)。

2.8. MI_MipiTx_InitDev

➤ 描述

初始化 MipiTx 设备

➤ 语法

```
MI_S32 MI_MipiTx_InitDev(MI\_MipiTx\_InitParam\_t *pstInitParam);
```

➤ 参数

参数名称	描述	输入/输出
pstInitParam	设备初始化参数。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

- 本接口必须和 [MI_MipiTx_DeInitDev](#) 成对使用，不可单独重复调用，否则返回失败。

➤ 举例

无

➤ 相关主题

无

2.9. MI_MipiTx_DeInitDev

➤ 描述

反初始化 MipiTx 设备

➤ 语法

```
MI_S32 MI_MipiTx_DeInitDev(void);
```

➤ 参数

无

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \text{ 成功。} \\ \text{非 } 0 \text{ 失败，参照[错误码](#)。} \end{array} \right.$

➤ 需求

- 头文件：mi_mipitx.h、mi_mipitx_datatype.h
- 库文件：libmi_mipitx.a / libmi_mipitx.so

※ 注意

- 此函数必须在初始化设备后调用，否则返回失败。
- 本接口必须和 [MI_MipiTx_InitDev](#) 成对使用，不可单独重复调用，否则返回失败。

➤ 举例

无

➤ 相关主题

无

3. MIPITX 数据类型

MipiTx 模块相关数据类型定义如下：

数据类型	定义
MI_MipiTx_TimingConfig_t	定义 MipiTx Timing 参数类型
MI_MipiTx_LaneNum_e	定义 MipiTx Lane num 的类型
MI_MipiTx_ChannelSwapType_e	定义 Lane Id 顺序
MI_MipiTx_ChannelAttr_t	定义 MipiTx Channel 通道属性
MI_MipiTx_InitParam_t	定义 MipiTx 设备初始化参数

3.1. MI_MipiTx_TimingConfig_t

➤ 说明

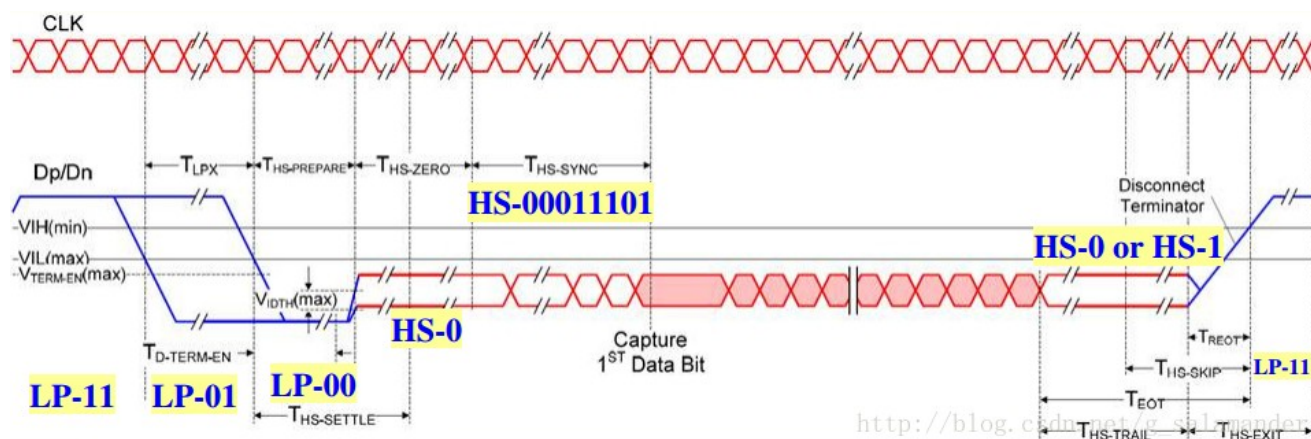
定义 MipiTx Timing 参数类型。

➤ 定义

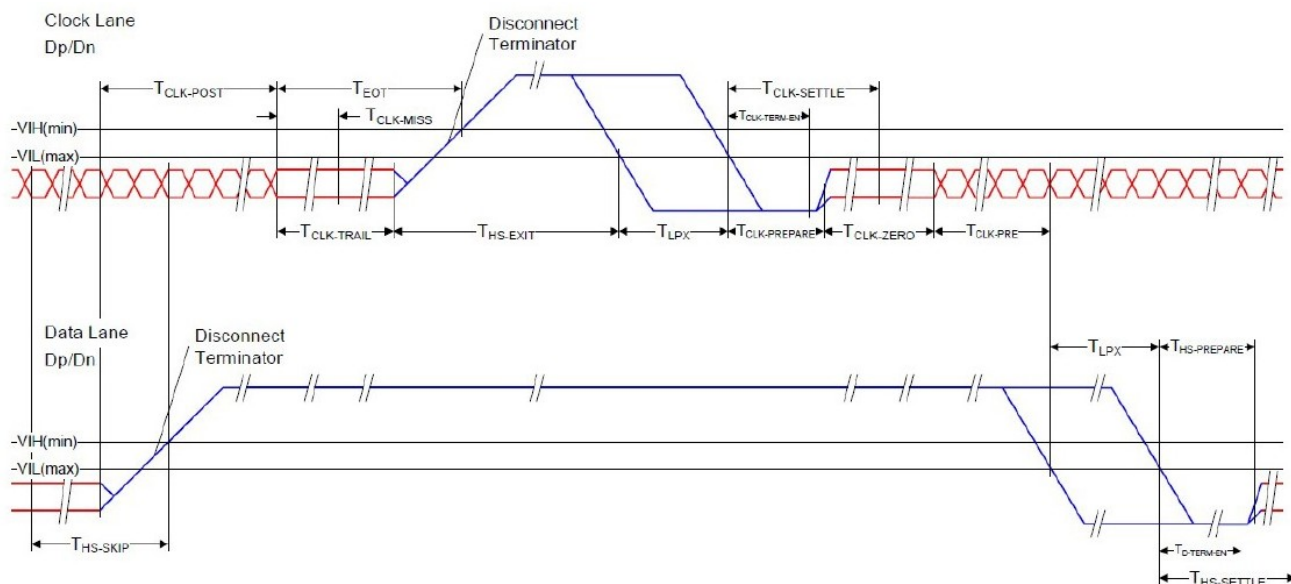
```
typedef struct MI_MipiTx_TimingConfig_s
{
    MI_U8 u8Lpx;
    MI_U8 u8ClkHsPrpr;
    MI_U8 u8ClkZero;
    MI_U8 u8ClkHsPre;
    MI_U8 u8ClkHsPost;
    MI_U8 u8ClkTrail;
    MI_U8 u8HsPrpr;
    MI_U8 u8HsZero;
    MI_U8 u8HsTrail;
} MI_MipiTx_TimingConfig_t;
```

➤ 成员

成员名称	描述
u8ClkHsPost	所有 data lane 进入低功耗模式后, Clock lane 保持高速时钟状态时间, 如图 3.1.2 中 $T_{CLK-POST}$
u8ClkTrail	进入低功耗模式前 Clock lane 维持 HS-0 时间, 如图 3.1.2 $T_{CLK-TRAIL}$
u8Lpx	低功耗模式 Clock lane 维持 LP-01 时间, 如图 3.1.2 T_{LPX}
u8ClkHsPrpr	低功耗模式 Clock lane 维持 LP-00 时间, 如图 3.1.2 $T_{CLK-PREPARE}$
u8ClkZero	进入高速模式前 Clock lane 维持 HS-0 时间, 如图 3.1.2 $T_{CLK-ZERO}$
u8ClkHsPre	Data 传输之前 Clock lane 维持高速时钟时间, 如图 3.1.2 $T_{CLK-PRE}$
u8HsPrpr	Data 传输之前 Data Lane 维持 LP-00 时间, 如图 3.1.1 $T_{HS-PREPARE}$
u8HsZero	Data 传输之前 Data Lane 维持 HS-0 时间, 如图 3.1.1 $T_{HS-ZERO}$
u8HsTrail	Data 传输结束后 Data Lane 维持 HS-0 或 HS-1, 如图 3.1.1 $T_{HS-TRAIL}$



由低功耗模式(LP)进入高速模式(HS)时序图 3.1.1



由高速模式进入低功耗模式时序图 3.1.2

※ 注意事项

Timing (T) [Ⓐ]	Default [Ⓐ]	Default [Ⓐ] @1Gbps [Ⓐ] (T _B = 8ns) [Ⓐ]	MIPI D-PHY Spec. (v1.2) [Ⓐ]		
			Min [Ⓐ]	Typ [Ⓐ]	Max [Ⓐ]
T _{LPX} [Ⓐ]	8T _B [Ⓐ]	64ns [Ⓐ]	50ns [Ⓐ]	Ⓐ	Ⓐ
T _{CLK-PREPARE} [Ⓐ]	9T _B [Ⓐ]	72ns [Ⓐ]	38ns [Ⓐ]	Ⓐ	95ns [Ⓐ]
T _{CLK-ZERO} + T _{CLK-PREPARE} [Ⓐ]	33T _B + 9T _B [Ⓐ]	336ns [Ⓐ]	300ns [Ⓐ]	Ⓐ	Ⓐ
T _{CLK-PRE} [Ⓐ]	2T _B [Ⓐ]	16ns [Ⓐ]	8ns [Ⓐ]	Ⓐ	Ⓐ
T _{CLK-POST} [Ⓐ]	17T _B [Ⓐ]	136ns [Ⓐ]	60ns + 52UI [Ⓐ]	Ⓐ	Ⓐ
T _{CLK-TRAIL} [Ⓐ]	11T _B [Ⓐ]	88ns [Ⓐ]	60ns [Ⓐ]	Ⓐ	Ⓐ
T _{HS-PREPARE} [Ⓐ]	10T _B [Ⓐ]	80ns [Ⓐ]	40ns + 4UI [Ⓐ]	Ⓐ	85ns + 6UI [Ⓐ]
T _{HS-ZERO} + T _{HS-PREPARE} [Ⓐ]	16T _B + 10T _B [Ⓐ]	208ns [Ⓐ]	145ns + 10UI [Ⓐ]	Ⓐ	Ⓐ
T _{HS-TRAIL} [Ⓐ]	11T _B [Ⓐ]	88ns [Ⓐ]	Max(8UI, 60ns + 4UI) [Ⓐ]	Ⓐ	Ⓐ

- 配置 Timing 可以参考上表，计算公式为 Time=(R+1)*TB，其中 Time 对应上表 Default 列，R 为所求项（对应结构体的值）。例如计算 u8Lpx，已知 Time=8，算得 R=7，所以 u8Lpx=7;

➤ 相关数据类型及接口

[MI_MipiTx_SetTimingConfig](#)

[MI_MipiTx_GetTimingConfig](#)

3.2. MI_MipiTx_LaneNum_e

➤ 说明

定义 MipiTx Lane num 的类型。

➤ 定义

```
typedef enum
{
    E_MI_MIPITX_LANE_NUM_NONE = 0,
    E_MI_MIPITX_LANE_NUM_1    = 1,
    E_MI_MIPITX_LANE_NUM_2    = 2,
    E_MI_MIPITX_LANE_NUM_3    = 3,
    E_MI_MIPITX_LANE_NUM_4    = 4,
} MI_MipiTx_LaneNum_e;
```

※ 注意事项

MipiTx 和 RX 的 Lane Num 数量一定相等。

➤ 相关数据类型及接口

[MI_MipiTx_ChannelAttr_t](#).

3.3. MI_MipiTx_ChannelSwapType_e

➤ 说明

定义 Lane Id 顺序。

➤ 定义

```
typedef enum
{
    E_MI_MIPITX_CH_SWAP_0,
    E_MI_MIPITX_CH_SWAP_1,
    E_MI_MIPITX_CH_SWAP_2,
    E_MI_MIPITX_CH_SWAP_3,
    E_MI_MIPITX_CH_SWAP_4,
} MI_MipiTx_ChannelSwapType_e;
```

※ 注意事项

默认情况下 lane0 CH0, lane1 CH1, lane2 CH2, lane3 CH3, Clk CH4 对应。

➤ 相关数据类型及接口

[MI_MipiTx_ChannelAttr_t](#).

3.4. MI_MipiTx_ChannelAttr_t

➤ 说明

定义 MipiTx Channel 通道属性。

➤ 定义

```
typedef struct
{
    MI_U32          u32Width;
    MI_U32          u32Height;
    MI_SYS_PixelFormat_e  ePixFormat;
    MI_MipiTx_LaneNum_e  eLaneNum;

    MI_U8          u8DCLKDelay; ///< DCLK Delay
    MI_U32          u32Dclk;    ///< DCLK ( Htt * Vtt * Fps)

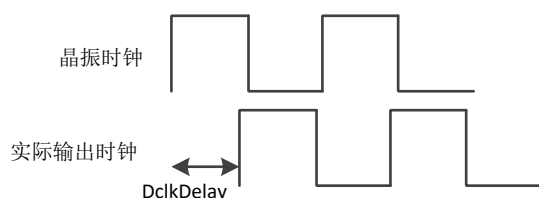
    MI_MipiTx_ChannelSwapType_e *peChSwapType; ///< all lane order swap
}MI_MipiTx_ChannelAttr_t;
```

➤ 成员

成员名称	描述
u32Width	传输数据水平大小
u32Height	传输数据垂直大小
ePixFormat	传输数据像素格式
enLaneNum	传输数据通道数量
u8DCLKDelay	传输数据时钟偏移
u32Dclk	传输数据时钟
peChSwapType	Lane id 排列顺序

※ 注意事项

- ePixFormat 当前只支持 YUV422 和 RAW8 格式。
- $u32Dclk = width * 1.2 * height * 1.2 * Fps * bit / LaneNum$ ，单位 HZ。
Dclk 有 4 个档位：100M~200M, 200M~400M, 400M~800M, 800M~1500M。
- u8DCLKDelay 默认为 0，不需要设置，预防以后调整时钟偏移



- 当 peChSwapType == NULL 时，chn0~4 和 lane 0~4 默认依次对应。否则，peChSwapType 需要指定对应于每个 CHN 的 lane Id。

➤ 相关数据类型及接口

[MI_MipiTx_CreateChannel](#)

3.5. MI_MipiTx_InitParam_t

➤ 说明

MipiTx 设备初始化参数。

➤ 定义

```
typedef struct MI_MipiTx_InitParam_s
```

```
{  
    MI_U32 u32DevId;  
    MI_U8 *u8Data;  
} MI_MipiTx_InitParam_t;
```

➤ 成员

成员名称	描述
u32DevId	设备ID
u8Data	数据buffer指针

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_MipiTx_InitDev](#)

4. 错误码

MipiTx API 错误码如表 3-1 所示:

表 3-1 MipiTx API 错误码

错误代码	宏定义	描述
0xA01E1016	MI_ERR_MIPITX_CHN_HAVECREATE	通道已经创建
0xA01E1002	MI_ERR_MIPITX_CHNID_INVALID	Channel Id 非法
0xA01E1014	MI_ERR_MIPITX_CHN_NOTSTOP	通道没有停止
0xA01E1003	MI_ERR_MIPITX_ILLEGAL_PARAM	非法参数
0xA01E2004	MI_ERR_MIPITX_RUNFAIL	功能执行失败
0xA01E2006	MI_ERR_MIPITX_NULL_PTR	输入参数空指针错误